

РОЗДІЛ 1. ОБРАННЯ ПРЕДМЕТНОЇ СИСТЕМИ ДЛЯ ПРОЄКТУВАННЯ

1.1. Визначення системи та предметної області

Для проєктування обрано *Інформаційну систему автоматизації бібліотечної діяльності* (далі - БІС). Предметною областю проєкту є сукупність процесів, що відбуваються всередині сучасної бібліотеки: від формування та каталогізації книжкового фонду до операційного обслуговування читачів та ведення обліку транзакцій видачі/повернення літератури.

Система розглядається як єдиний інформаційний простір, що об'єднує фізичні активи (книги), користувачів та управлінський персонал (бібліотекарів та адміністраторів) за допомогою цифрових інтерфейсів.

Ключові параметри системи:

- **Мета системи:** створення інтегрованого програмного середовища для автоматизації рутинних операцій, мінімізації помилок ручного введення даних, забезпечення прозорого контролю за станом фонду та надання користувачам сучасного сервісу онлайн-резервування.
- **Очікувані користувачі:** Читачі (студенти/відвідувачі), Бібліотекарі (оператори фонду) та Адміністратори (керівники системи та аналітики).

1.2. Обґрунтування вибору предметної області

Вибір бібліотечної системи зумовлений її високою репрезентативністю для вивчення системного аналізу, оскільки вона містить усі типи системних зв'язків та циклів.

Актуальність. У сучасних умовах цифровізації традиційні методи обліку (паперові картки, журнали) стають неефективними. Аналіз існуючих рішень показав, що більшість застарілих систем мають архаїчний інтерфейс та низьку юзабіліті, що відштовхує сучасного користувача. Автоматизація бібліотеки дозволяє не лише зберегти фонд, а й підвищити рівень інформаційної культури закладу.

Складність проєктування. Проєкт характеризується необхідністю реалізації складних архітектурних та логічних рішень:

- **Трирівнева архітектура (Multi-Tier):** Поділ на рівень представлення (React), бізнес-логіки (Node.js) та даних (PostgreSQL), що вимагає чіткого проєктування інтерфейсів взаємодії.
- **Транзакційна логіка:** Необхідність обробки станів об'єктів (наприклад, перехід книги зі статусу «Доступна» у «Видана» з одночасним розрахунком дати повернення).
- **Системна аналітика:** Побудова моделей для відстеження заборгованостей та візуалізації статистики обігу фонду.

Доцільність автоматизації. Впровадження БІС дозволяє вирішити наступні критичні проблеми:

- **Надійність обліку:** Використання реляційної бази даних усуває ризики втрати чи фізичного пошкодження інформації.
- **Проактивний контроль:** Система самостійно ідентифікує протерміновані видачі та формує сигнали для адміністратора, що неможливо при ручному обліку.
- **Оптимізація ресурсів:** Бібліотекарі звільняються від рутинної паперової роботи, фокусуючись на якісному наповненні каталогу та консультуванні читачів.

РОЗДІЛ 2. ЗБІР І АНАЛІЗ ВИМОГ

2.1. Обстеження предметної області та аналіз бізнес-процесів

Метою даного етапу є детальне вивчення внутрішніх механізмів функціонування бібліотеки для їх подальшої цифрової трансформації. Об'єктом дослідження є інформаційні потоки, що виникають між читачем, книжковим фондом та персоналом.

У ході обстеження було виділено три ключові бізнес-процеси, що потребують повної автоматизації:

1. **Централізований облік та каталогізація:** створення єдиного реєстру книг із детальним описом (автор, назва, ISBN) та прив'язкою до конкретних фізичних примірників з унікальними інвентарними номерами.
2. **Управління обігом (Circulation):** фіксація моменту передачі книги читачеві, зміна її статусу в реальному часі та встановлення жорстких термінів повернення.
3. **Моніторинг та зворотний зв'язок:** автоматичне виявлення заборгованостей та інформування адміністратора про необхідність вжиття заходів щодо повернення активів бібліотеки.

2.2. Специфікація функціональних вимог (Functional Requirements - FR)

Функціональні вимоги визначають, що саме система повинна робити для задоволення потреб користувачів.

ID	Категорія	Опис вимоги
FR-01	Автентифікація	Система повинна забезпечувати безпечний вхід через email та пароль з розмежуванням ролей (Admin, Librarian, Reader).
FR-02	Каталог	Читач повинен мати можливість здійснювати динамічний пошук за назвою та фільтрувати книги за категоріями.
FR-03	Резервування	Система повинна дозволяти онлайн-бронювання лише тих примірників, що мають статус "Available".
FR-04	Видача	Бібліотекар повинен мати інтерфейс для реєстрації видачі, де автоматично розраховується due_date (дата повернення).
FR-05	Аналітика	Адміністратор повинен бачити статистику фонду (видані/доступні книги) у вигляді інтерактивних графіків.

Табл. 1 "Функціональні вимоги"

2.3. Нефункціональні вимоги (Non-Functional Requirements - NFR)

Ці вимоги визначають якісні характеристики системи та технічні обмеження.

- **Архітектурна цілісність:** Система повинна базуватися на трирівневій моделі (Presentation, Application, Data Layer) для забезпечення незалежності компонентів та масштабованості.
- **Надійність та безпека:** Всі чутливі дані (паролі) мають зберігатися в зашифрованому вигляді, а доступ до API повинен бути захищений JWT-токенами.
- **Юзабіліті (Usability):** Користувацький інтерфейс повинен бути адаптивним, з часом відгуку на запит пошуку не більше 2 секунд.
- **Конфігураційність:** Параметри підключення до бази даних та секретні ключі повинні вноситися в окремі змінні оточення (.env), що дозволяє легко змінювати середовище розгортання.

2.4. Обмеження та зовнішні умови

- **Технологічний стек:** Реалізація повинна базуватися на Full Stack JavaScript (React для фронтенду, Node.js для бекенду).
- **База даних:** Використання реляційної СУБД PostgreSQL для гарантування цілісності транзакцій.
- **Бюджет та терміни:** Проєкт реалізується в рамках навчального курсу, що обмежує використання платних сторонніх сервісів.

РОЗДІЛ 3. ВИЗНАЧЕННЯ КОРИСТУВАЧІВ СИСТЕМИ

3.1. Ролі та функціональне призначення

Для забезпечення чіткого розмежування прав доступу, безпеки даних та зручності експлуатації в інформаційній системі визначено три основні групи (класи) користувачів. Кожна роль має свій унікальний набір повноважень та зону відповідальності в межах бізнес-логіки бібліотеки.

Роль (Клас)	Рівень доступу	Функції та відповідальність
Читач (Reader)	Базовий	Самостійна реєстрація, автентифікація, перегляд актуального каталогу книг, фільтрація за категоріями та онлайн-резервування доступних примірників.
Бібліотекар (Librarian)	Операційний	Ведення обліку книжкового фонду, реєстрація видачі книг читачам (фіксація <code>issue_date</code> та <code>due_date</code>), прийом повернених книг, контроль фізичного стану активів.
Адміністратор (Admin)	Системний	Повний контроль над системою: керування обліковими записами, зміна ролей, моніторинг роботи сервісів нагадувань, аналіз графіків

		обігу та формування звітності про заборгованості.
--	--	---

Табл. 2 “Ролі”

3.2. Матриця доступу до функцій системи (RBAC)

Система базується на моделі керування доступом на основі ролей (Role-Based Access Control). Це дозволяє гнучко налаштовувати права доступу на рівні Middleware бекенду.

- **Рівень представлення (UI):**

- Читач бачить лише публічний каталог та сторінку «Мої резервування».
- Бібліотекар має доступ до розширеного інтерфейсу керування екземплярами книг та журналом видач.
- Адміністратор має доступ до спеціалізованої панелі «Dashboard» з аналітичними графіками.

- **Рівень даних (API):**

- Маршрути /api/admin/* закриті для всіх, крім ролі Admin.
- Маршрути створення видач /api/loans доступні лише ролям Librarian та Admin.
- Читач може змінювати лише власні дані профілю та статуси своїх бронювань.

3.3. Обмеження безпеки та права доступу

Забезпечення конфіденційності та цілісності даних є пріоритетом системи:

- **Автентифікація:** Доступ до будь-яких персоналізованих функцій надається лише після успішної перевірки JWT-токена, який генерується при вході.
- **Ізоляція даних:** Користувачі ролі «Читач» не мають прямого доступу до бази даних та не можуть бачити списки інших користувачів або історію їхніх видач.
- **Контроль сесій:** Для запобігання несанкціонованому використанню акаунту, термін дії токена обмежений у часі (1 година), після чого потрібна повторна авторизація.

РОЗДІЛ 4. ФУНКЦІОНАЛЬНІ СЦЕНАРІЇ (USER STORIES)

4.1. Принципи формування функціональних сценаріїв

Функціональні сценарії або User Stories є інструментом гнучкого моделювання, що дозволяє описати можливості системи з точки зору кінцевого користувача. Вони зосереджені на цінності, яку отримує актор при взаємодії з інтерфейсом, і слугують основою для подальшого проектування логіки API та структури бази даних.

Для проекту сценарії охоплюють повний життєвий цикл - від первинної зацікавленості читача книгою до адміністративного контролю за поверненням активів.

4.2. Перелік User Stories за ролями

Сценарії для ролі «Читач» (Reader)

- **Пошук та фільтрація:** «Як Читач, я хочу використовувати рядок пошуку та фільтри за категоріями в каталозі, щоб швидко знайти потрібну літературу серед великої кількості книг».
- **Онлайн-резервування:** «Як Читач, я хочу мати можливість зарезервувати вибрану книгу через свій особистий кабінет, щоб бути впевненим, що примірник не буде виданий іншому користувачу до мого візиту в бібліотеку».
- **Контроль власних видач:** «Як Читач, я хочу бачити список своїх активних позик та дати їх повернення, щоб уникнути штрафних санкцій та вчасно повернути літературу».

Сценарії для ролі «Бібліотекар» (Librarian)

- **Реєстрація видачі:** «Як Бібліотекар, я хочу зафіксувати видачу книги читачеві, вказавши його ID та інвентарний номер книги, щоб система автоматично змінила статус примірника на "Issued" та розрахувала крайній термін повернення».
- **Прийом повернення:** «Як Бібліотекар, я хочу відмітити повернення книги у системі, щоб вона миттєво знову стала доступною для пошуку іншими читачами в загальному каталозі».

Сценарії для ролі «Адміністратор» (Administrator)

- **Моніторинг заборгованостей:** «Як Адміністратор, я хочу запускати автоматичну перевірку термінів (overdue-check), щоб отримати актуальний список боржників та ініціювати процес їх сповіщення».
- **Системна аналітика:** «Як Адміністратор, я хочу переглядати візуалізовані звіти про стан фонду та активність читачів за допомогою графіків, щоб приймати обґрунтовані рішення щодо закупівлі нових книг».

4.3. Критерії прийнятності (Acceptance Criteria)

Кожна історія вважається реалізованою лише за умови виконання технічних вимог:

- Система повинна миттєво оновлювати стан бази даних при зміні статусу примірника.
- При спробі зарезервувати вже зайняту книгу, користувач має отримати чітке повідомлення про помилку.
- Дані про термін повернення повинні розраховуватися автоматично на основі поточної дати та встановлених бізнес-правил системи.

РОЗДІЛ 5. ВІДОБРАЖЕННЯ USER STORIES У UML-НОТАЦІЇ

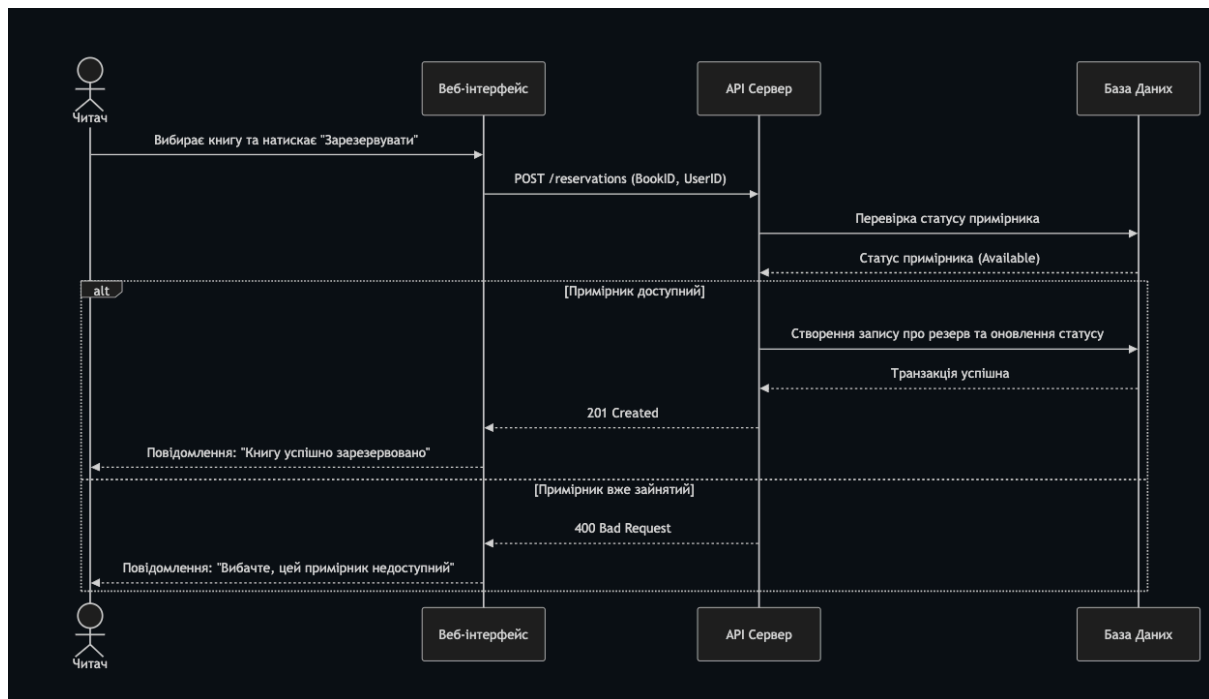
Для візуалізації динаміки взаємодії між компонентами системи та користувачами використовуються **діаграми послідовності (Sequence Diagrams)**. Вони дозволяють детально описати порядок обміну повідомленнями, виклики API та транзакції в базі даних для конкретних функціональних сценаріїв.

5.1. Діаграма послідовності: Резервування книги Читачем

Цей сценарій описує процес, коли Читач обирає книгу в каталозі та ініціює її онлайн-бронювання. Важливою частиною логіки є перевірка статусу примірника перед створенням запису в таблиці резервувань.

Опис процесу:

- **Reader -> UI:** Вибір книги та натискання кнопки «Зарезервувати».
- **UI -> API:** Виклик методу POST /reservations з ідентифікаторами користувача та книги.
- **API -> DB:** Перевірка наявності доступних примірників (is_available = true).
- **DB -> API:** Повернення результату перевірки.
- **API (Логіка):** Якщо примірник знайдено, статус змінюється на Reserved, створюється запис у таблиці Reservations.
- **API -> UI:** Повернення коду успіху (201 Created) або помилки (400 Bad Request).



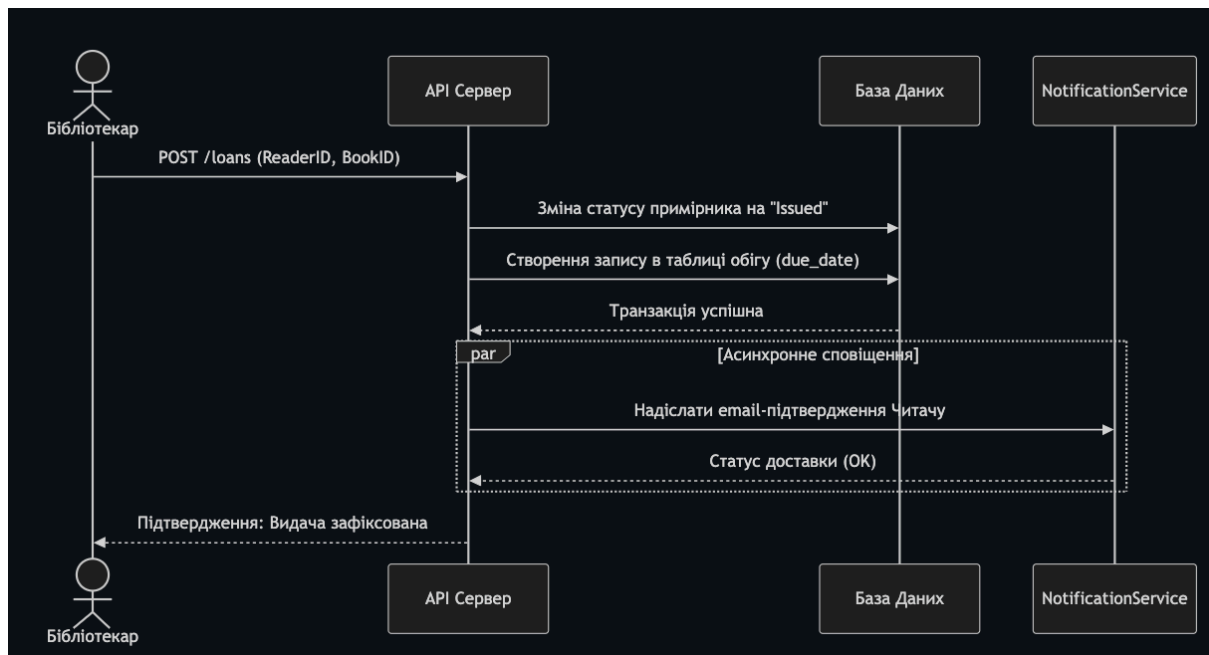
Мал. №1

5.2. Діаграма послідовності: Видача книги Бібліотекарем

Сценарій охоплює фізичну видачу книги в приміщенні бібліотеки. Система автоматизує розрахунок дати повернення та інтегрується із сервісом сповіщень для підтвердження операції.

Опис процесу:

- **Librarian -> API:** Запит на створення видачі (POST /loans).
- **API -> DB:** Оновлення статусу примірника на Issued та фіксація due_date.
- **API -> NotificationService:** Асинхронний виклик методу sendDueReminder для підтвердження видачі на email Читача.
- **API -> Librarian:** Візуальне підтвердження успішного завершення операції в інтерфейсі.



Мал. №2

5.3. Аналіз альтернативних сценаріїв

У процесі побудови діаграм враховано альтернативні гілки (Alt-blocks):

- **Відсутність примірників:** Якщо при запиті на резервування база даних повертає порожній результат, API перериває процес і надсилає повідомлення про недоступність книги.
- **Помилка автентифікації:** Якщо JWT-токен невалідний, Middleware блокує запит до БД і повертає статус 401.

РОЗДІЛ 6. АРХІТЕКТУРА СИСТЕМИ ТА ЗАЛЕЖНОСТІ МІЖ ПІДСИСТЕМАМИ

6.1. Логічна архітектурна модель (Three-Tier Architecture)

Проект інформаційної системи базується на класичній **трирівневій архітектурі**, що забезпечує високу модульність, незалежність компонентів та полегшує процес масштабування й тестування кожної підсистеми окремо.

1. Рівень представлення (Presentation Layer):

- Реалізований за допомогою бібліотеки **React**.
- Відповідає за візуалізацію даних, взаємодію з користувачем через форми та динамічне оновлення інтерфейсу за допомогою стану (State Management).
- Взаємодіє з сервером через асинхронні HTTP-запити (Axios).

2. Рівень бізнес-логіки (Application/Business Logic Layer):

- Побудований на базі середовища **Node.js** та фреймворку **Express**.
- Містить основні алгоритми: перевірку прав доступу (RBAC), обробку транзакцій видачі, розрахунок термінів повернення та ініціалізацію сервісів сповіщень.
- Виступає посередником між клієнтом та базою даних, забезпечуючи валідацію вхідних даних.

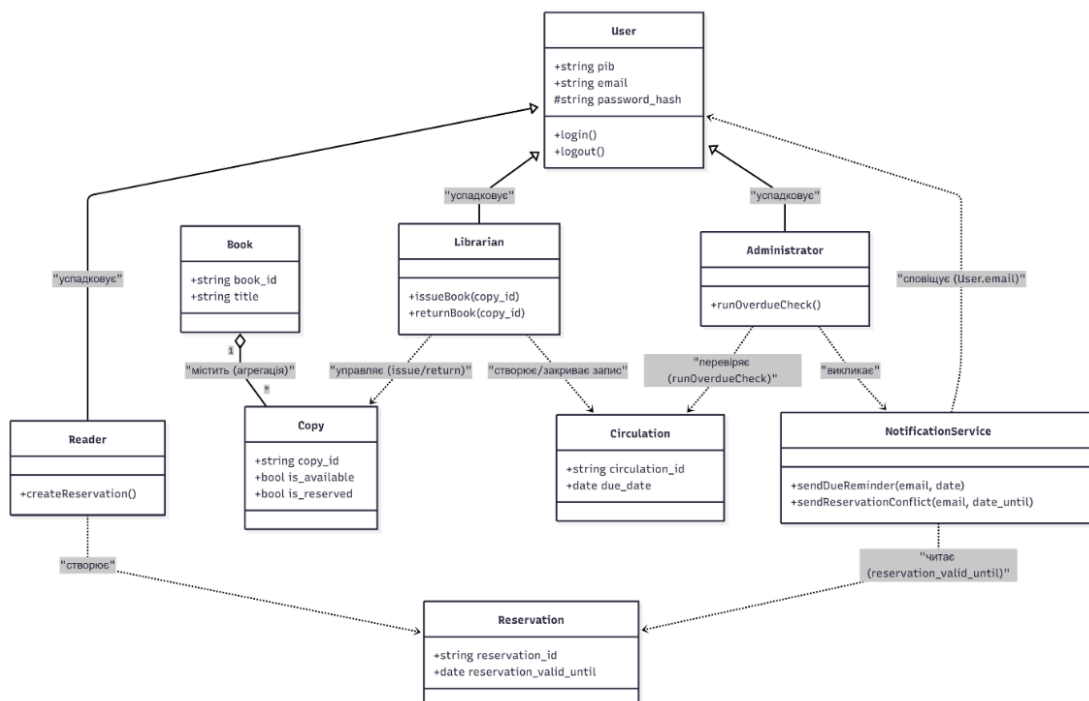
3. Рівень даних (Data Layer):

- Використовує реляційну СУБД **PostgreSQL**.
- Забезпечує персистентність (тривале зберігання) інформації про книги, користувачів та історію обігу.
- Контролює цілісність даних через систему зовнішніх ключів та транзакцій.

6.2. Діаграма компонентів та взаємозв'язків

Діаграма компонентів відображає структурну організацію програмних модулів та їхні залежності.

- **Web Client (React):** містить компоненти авторизації, каталогу та адмін-панелі.
- **API Server (Express):** поділений на маршрути (routes), контролери (controllers) та посередники (middleware).
- **Database (PostgreSQL):** взаємодіє з сервером через драйвер pg або ORM-систему (Sequelize).
- **External Service (Email/Notification):** зовнішня підсистема, що використовується для надсилання асинхронних повідомлень читачам.



Мал. №3

6.3. Програмна реалізація точки входу (server.js)

Центральним вузлом архітектури бекенду є файл server.js, який координує запуск усіх підсистем.

JavaScript

```
const express = require('express');

const cors = require('cors');

const authRoutes = require('./routes/auth');

const libraryRoutes = require('./routes/library');

const { sequelize } = require('./models');

const app = express();

app.use(cors()); // Забезпечення безпеки міждомених запитів

app.use(express.json());

// Підключення модулів бізнес-логіки

app.use('/api/auth', authRoutes);

app.use('/api/library', libraryRoutes);

// Запуск сервера та синхронізація з БД

const PORT = process.env.PORT || 5000;

sequelize.sync().then(() => {

  app.listen(PORT, () => console.log(`Сервер працює на порту ${PORT}`));

});
```

Даний фрагмент коду демонструє інтеграцію рівнів та використання проміжних обробників для забезпечення функціональності системи.

РОЗДІЛ 7. ПОБУДОВА МОДЕЛІ ДАНИХ

7.1. Основні сутності та атрибутивний склад

Проектування моделі даних для системи базується на принципах нормалізації для забезпечення цілісності та мінімізації надмірності інформації. Визначено чотири базові сутності, які відображають структуру бібліотечного фонду та логіку взаємодії з користувачами:

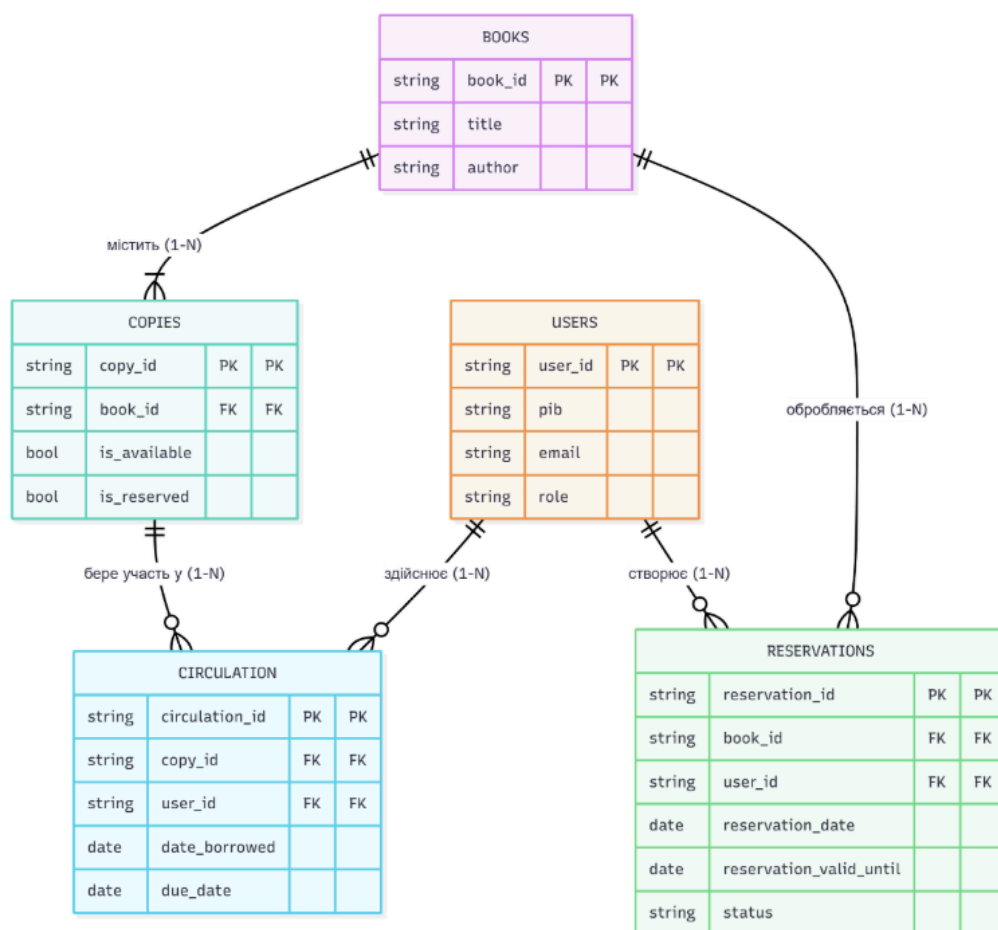
- **Користувачі (Users):** Зберігає персональні дані, контактну інформацію та системну роль (Admin, Librarian, Reader).
 - *Атрибути:* user_id (PK), full_name, email, password_hash, role.
- **Книги (Books):** Містить загальний опис літературного твору.
 - *Атрибути:* book_id (PK), title, author, isbn, category.
- **Примірники (Book_Items):** Відображає фізичні одиниці зберігання. Одна книга може мати багато примірників.
 - *Атрибути:* item_id (PK), book_id (FK), inventory_number, status (Available, Issued, Reserved).
- **Обіг/Видачі (Loans):** Фіксує транзакції передачі книги користувачеві.
 - *Атрибути:* loan_id (PK), user_id (FK), item_id (FK), issue_date, due_date, return_date.

7.2. ER-діаграма та логічні зв'язки

Зв'язки між сутностями побудовані за реляційною логікою «один-до-багатьох» (1:N):

1. **Book - Book_Items (1:N):** Кожна книга в каталозі може бути представлена декількома фізичними копіями з різними інвентарними номерами.

2. **User - Loans (1:N)**: Один користувач може мати декілька записів про видачу книг протягом часу.
3. **Book_Items - Loans (1:N)**: Конкретний примірник може фігурувати у багатьох записах про видачу (історія обігу), але в конкретний момент часу він може бути прив'язаний лише до однієї активної видачі.



Мал. №4

7.3. Фізична реалізація схеми (SQL)

Реалізація схеми в PostgreSQL передбачає використання обмежень цілісності (NOT NULL, UNIQUE) та каскадних оновлень для підтримки актуальності даних.

SQL

-- Створення таблиці книг

```
CREATE TABLE books (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    author VARCHAR(255),  
    isbn VARCHAR(20) UNIQUE  
);
```

-- Створення таблиці примірників з посиланням на книгу

```
CREATE TABLE book_items (  
    id SERIAL PRIMARY KEY,  
    book_id INTEGER REFERENCES books(id) ON DELETE CASCADE,  
    inventory_number VARCHAR(50) UNIQUE,  
    status VARCHAR(20) DEFAULT 'Available'  
);
```

-- Створення таблиці обліку видач

```
CREATE TABLE loans (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER REFERENCES users(id),  
    item_id INTEGER REFERENCES book_items(id),
```

```
issue_date DATE DEFAULT CURRENT_DATE,  
due_date DATE NOT NULL,  
return_date DATE  
);
```

7.4. Відповідність моделі сценаріям використання

Дана модель даних повністю підтримує User Stories, описані в Розділі 4:

- Для **пошуку** використовується таблиця books.
- Для **резервування** та **видачі** змінюється поле status у book_items та створюється запис у loans.
- Для **аналітики** адміністратор виконує агрегаційні запити до loans для підрахунку кількості виданих книг та виявлення протермінувань.

РОЗДІЛ 8. ОБҐРУНТУВАННЯ ВИБОРУ ТИПУ БАЗИ ДАНИХ

Для реалізації інформаційної системи автоматизації бібліотеки обрано **об'єктно-реляційну СУБД PostgreSQL**. Даний вибір є стратегічним рішенням, що ґрунтується на аналізі характеру даних, вимог до цілісності транзакцій та необхідності побудови складних аналітичних звітів.

8.1. Відповідність реляційної моделі структурі даних

Згідно з архітектурним планом (Розділ 6), система оперує великою кількістю чітко структурованих та взаємопов'язаних сутностей. Реляційна модель PostgreSQL дозволяє реалізувати:

- **Нормалізація даних:** Структура таблиць розроблена для мінімізації надмірності. Поділ на загальну інформацію про видання (books) та конкретні фізичні одиниці (book_items) дозволяє уникнути дублювання текстових даних, що критично для великих фондів.
- **Суворі типізація:** Використання чітких типів даних (INTEGER, VARCHAR, DATE) запобігає появі логічних помилок на етапі запису інформації.
- **Цілісність посилань (Referential Integrity):** Обмеження FOREIGN KEY гарантують, що жодна видача (loan) не може бути створена без посилання на існуючого читача та конкретну книгу.

8.2. Обробка транзакцій та надійність (ACID)

У бібліотечних процесах критично важливою є атомарність операцій. Наприклад, при видачі книги система повинна одночасно змінити статус примірника на "Issued" та створити новий запис у таблиці обігу.

- **ACID-відповідність:** PostgreSQL гарантує, що якщо будь-яка частина транзакції (наприклад, запис у `loans`) зазнає невдачі, всі попередні зміни (наприклад, зміна статусу в `book_items`) будуть скасовані (Rollback).
- **Ізоляція:** Це запобігає виникненню ситуацій, коли два бібліотекарі одночасно намагаються видати один і той самий примірник різним читачам.

8.3. Програмна реалізація та продуктивність

- **Механізм pg-pool:** У коді бекенду (файл `db.js`) впроваджено пул з'єднань, що дозволяє системі ефективно перерозподіляти ресурси сервера при великій кількості одночасних запитів до каталогу.
- **Складні запити (Complex JOINS):** Для реалізації модуля аналітики (Розділ 13) PostgreSQL забезпечує високу швидкість виконання агрегаційних запитів, що поєднують дані з декількох таблиць для формування звітів у реальному часі.

JavaScript

// Приклад логіки підключення до PostgreSQL (db.js)

```
const { Pool } = require('pg');  
  
const pool = new Pool({  
  user: process.env.DB_USER,  
  host: process.env.DB_HOST,  
  database: process.env.DB_NAME,  
  password: process.env.DB_PASSWORD,  
  port: process.env.DB_PORT,  
});  
  
module.exports = { query: (text, params) => pool.query(text, params), };
```

РОЗДІЛ 9. АНАЛІЗ МОЖЛИВОСТЕЙ МАСШТАБУВАННЯ СИСТЕМИ

Система спроектована за модульним принципом «клієнт-сервер», що створює фундамент для обох типів масштабування при зростанні кількості користувачів або обсягу книжкового фонду. Чітке розділення на Presentation, Application та Data Layer дозволяє масштабувати кожен рівень незалежно.

9.1. Вертикальне масштабування (Scaling Up)

Вертикальне масштабування передбачає збільшення обчислювальних потужностей поточного сервера (CPU, RAM).

- **Асинхронність Node.js:** Використання неблокуючого вводу-виводу дозволяє серверу обробляти тисячі одночасних з'єднань на одному ядрі процесора.
- **Кластеризація (Node.js Cluster):** Для повної утилізації багатоядерних процесорів реалізовано можливість запуску окремих воркерів (потоків) на кожне ядро, що кратно збільшує продуктивність API.
- **Оптимізація БД:** PostgreSQL дозволяє розширювати обсяг кешування (shared_buffers) для швидкої обробки SQL-запитів у великих таблицях історії позик.

9.2. Горизонтальне масштабування (Scaling Out)

Цей підхід передбачає додавання нових вузлів у систему для розподілу навантаження.

- **Розподілена інфраструктура:** Оскільки React-фронтенд та Node.js-API роз'єднані, вони можуть бути розгорнуті на різних серверах або в хмарних контейнерах (Docker).
- **Балансування навантаження (Load Balancing):** Впровадження Nginx як реверс-проксі дозволяє розподіляти вхідний трафік між декількома примірниками API-сервера, забезпечуючи відмовостійкість.
- **Кешування аналітики:** Оскільки система використовує Chart.js для візуалізації (функція `getStats`), при великій кількості запитів доцільно впровадити Redis. Це дозволить зберігати результати складних агрегаційних запитів у пам'яті, знімаючи навантаження з основної СУБД.

9.3. Перехід до мікросервісів та хмарна архітектура

Поточна монолітна структура бекенду спроектована так, що окремі модулі можна винести в мікросервіси:

- **Сервіс сповіщень:** Модуль `overdue-check` та `NotificationService` можуть функціонувати як окремий Worker-сервіс, що працює за розкладом.
- **Хмарна БД:** База даних PostgreSQL може бути перенесена на сервіси на кшталт AWS RDS, що забезпечує автоматичне резервування та реплікацію даних.

РОЗДІЛ 10. ПРОАНАЛІЗУВАТИ СИСТЕМУ БЕЗПЕКИ

Безпека інформаційної системи побудована на принципах ешелонованого захисту, що охоплює рівень передачі даних, рівень автентифікації користувачів та рівень розмежування доступу до ресурсів бази даних.

10.1. Автентифікація та криптографічний захист паролів

Система відмовляється від зберігання паролів у відкритому вигляді, що є критичною вимогою для сучасних веб-додатків.

- **Алгоритм Bcryptjs:** При реєстрації або зміні пароля використовується бібліотека `bcryptjs` для створення стійкого до перебору хешу з використанням «солі» (`salt`). Це гарантує, що навіть у разі витоку бази даних, зломисники не зможуть відновити оригінальні паролі користувачів.
- **Процес перевірки:** Під час входу система хешує введений пароль і порівнює його з хешем, що зберігається в БД. Весь процес відбувається на стороні сервера, що виключає передачу конфіденційних даних у відкритому виді через клієнтські скрипти.

10.2. Авторизація на основі JWT (JSON Web Tokens)

Для підтримки стану авторизації без зберігання сесій на сервері (Stateless) використано стандарт JWT.

- **Генерація токена:** Після успішного входу сервер генерує токен, що містить `user_id` та `role`. Токен підписується секретним ключем, який зберігається в змінній оточення `JWT_SECRET`.
- **Життєвий цикл:** Токен має обмежений термін дії (наприклад, 1 година), що мінімізує ризики у разі його перехоплення.

- **Перевірка (Middleware):** Кожен захищений запит до API (наприклад, видача книги) проходить крізь посередник `authenticateToken`, який розшифровує токен і перевіряє його валідність перед тим, як надати доступ до логіки контролера.

10.3. Управління правами доступу (RBAC)

Для захисту бізнес-функцій реалізовано механізм перевірки ролей (Role-Based Access Control).

- **Публічні маршрути:** Пошук у каталозі та перегляд інформації про книги доступні всім авторизованим користувачам.
- **Захищені маршрути:** Функції `createLoan` або `updateInventory` захищені спеціальним Middleware `authorizeRole(['Librarian', 'Admin'])`. Якщо роль користувача в токені не відповідає дозволеним, система повертає код помилки 403 Forbidden.

10.4. Захист від поширених вразливостей

- **SQL Injection:** Використання параметризованих запитів у драйвері `pg` (наприклад, `query('SELECT * FROM users WHERE id = $1', [id])`) повністю виключає можливість ін'єкцій шкідливого коду в SQL-команди.
- **CORS (Cross-Origin Resource Sharing):** Налаштування політики CORS обмежує доступ до API лише з довірених доменів (наприклад, тільки з фронтенд-адреси), що запобігає запитам від сторонніх зловмисних сайтів.
- **Environment Variables:** Всі чутливі дані (паролі до БД, секрети JWT) зберігаються у файлі `.env`, який внесено до `.gitignore`, щоб запобігти їх потраплянню у відкритий репозиторій.

РОЗДІЛ 11. ВИЗНАЧЕННЯ ПЕРСПЕКТИВ РОЗВИТКУ СИСТЕМИ

Інформаційна система спроектована як еволюційний артефакт, що має високий потенціал для розширення функціоналу та інтеграції з новими технологічними платформами. На основі аналізу поточного стану та лекційних матеріалів щодо розширюваності інтеграційних інтерфейсів, визначено наступні стратегічні напрямки розвитку:

11.1. Впровадження технологій Internet of Things (IoT)

Для повної автоматизації фізичного рівня обігу книг планується перехід від штрих-кодування до **RFID-ідентифікації** (Radio Frequency Identification).

- **Автоматизована інвентаризація:** використання ручних RFID-зчитувачів дозволить проводити перевірку всього фонду за лічені години без зняття книг з полиць.
- **Станції самообслуговування:** читачі зможуть самостійно реєструвати видачу та повернення книг, просто поклавши їх на спеціальну панель, що значно розвантажить персонал.

11.2. Розширення клієнтської бази та мобільна доступність

Поточна веб-орієнтована архітектура дозволяє легко масштабувати інтерфейсну частину для різних пристроїв.

- **Розробка мобільного додатка:** створення кросплатформеного додатка (на базі React Native), що надасть читачам доступ до цифрового квитка, Push-сповіщень про терміни повернення та можливості сканування ISBN для швидкого пошуку.

- **PWA (Progressive Web App):** трансформація існуючого фронтенду в прогресивний веб-додаток для забезпечення роботи в офлайн-режимі та швидкого доступу з головного екрана смартфона.

11.3. Інтеграція з зовнішніми сервісами (API First)

Згідно з принципами розширюваності інтерфейсів, система має стати частиною ширшої освітньої екосистеми.

- **Інтеграція з Google Books API:** автоматичне завантаження анотацій, обкладинок та рейтингів книг для збагачення локального каталогу.
- **Telegram/Viber Bot:** впровадження чат-бота для миттєвого інформування боржників про наближення дати повернення (due_date) та автоматичного продовження терміну оренди.

11.4. Впровадження алгоритмів штучного інтелекту (AI)

- **Рекомендаційна система:** аналіз історії читань для надання персоналізованих рекомендацій на основі вподобань користувача.
- **Прогнозування попиту:** використання методів машинного навчання для аналізу популярності певних жанрів чи авторів, що дозволить адміністрації більш ефективно планувати бюджет на закупівлю нових примірників.

11.5. Безпека та хмарна інфраструктура

- **Blockchain-захист:** розгляд можливості використання блокчейн-технологій для незмінного зберігання історії рідкісних видань та цінних примірників.
- **Multi-Cloud Deployment:** перехід на хмарні сервіси (AWS або Google Cloud) для забезпечення автоматичного резервування та географічного розподілу даних.

РОЗДІЛ 12. ОБРАННЯ МЕТОДОЛОГІЇ РОЗРОБКИ

12.1. Аргументація вибору гнучкої методології (Agile)

Для реалізації проєкту інформаційної системи було обрано **ітеративно-інкрементальний підхід** на базі методології **Agile (фреймворк Scrum)**. Даний вибір зумовлений високим рівнем невизначеності на початкових етапах та необхідністю постійного тестування гіпотез щодо зручності інтерфейсу для різних ролей користувачів.

Переваги обраного підходу для БІС:

- **Рання візуалізація:** можливість продемонструвати базовий функціонал каталогу ще до завершення розробки модуля аналітики.
- **Адаптивність до змін:** гнучка зміна пріоритетів (Backlog) дозволила зосередитися на безпеці (JWT, bcrypt) після виявлення потенційних вразливостей у схемі авторизації.
- **Зниження ризиків:** кожен спринт закінчувався робочим інкрементом коду, що гарантувало цілісність системи на кожному етапі.

12.2. Структура життєвого циклу розробки (SDLC)

Процес розробки було розділено на декілька ключових ітерацій (спринтів), кожна з яких тривала 2 тижні:

1. **Спринт 1: Фундамент (Core & Data Layer):** Проєктування ER-діаграми, налаштування PostgreSQL та ініціалізація Express-сервера.
2. **Спринт 2: Безпека та Профілі (Auth & User Management):** Реалізація реєстрації, хешування паролів та видача JWT-токенів.

3. **Спринт 3: Операційний блок (Circulation Logic):** Створення механізмів видачі та повернення книг, зміна статусів у `book_items`.
4. **Спринт 4: Інтерфейс та Аналітика (UI & Analytics):** Розробка React-компонентів, інтеграція `Chart.js` та фінальне тестування сценаріїв.

12.3. Узгодження методології з архітектурою системи

Використання Scrum ідеально корелює з трирівневою архітектурою (Розділ 6). Оскільки шари системи (Frontend, Backend, Database) є незалежними, команда мала змогу паралельно працювати над SQL-схемами та UI-компонентами, використовуючи API як контракт взаємодії.

12.4. Командна організація та інструментарій

Для підтримки методології використовувалися сучасні інструменти контролю версій та управління завданнями:

- **Git/GitHub:** для ведення гілок розробки та фіксації окремих комітів для кожного модуля (Розділи 6, 7, 10...).
- **Trello/Jira:** для візуалізації руху завдань від "To Do" до "Done".
- **Postman:** для ітеративного тестування endpoint-ів API перед їх інтеграцією з фронтом.

Даний підхід дозволив створити не просто статичний програмний код, а життєздатну систему, яка готова до подальшого масштабування та еволюції (Розділ 11).

РОЗДІЛ 13. ДОКУМЕНТУВАННЯ РЕЗУЛЬТАТІВ ПРОЄКТУ

13.1. Загальний огляд реалізованого рішення

Результатом виконання проєкту є повнофункціональна інформаційна система, що забезпечує повний життєвий цикл управління бібліотечним фондом. Проєктна документація відображає інтеграцію всіх попередніх етапів: від аналізу вимог до розгортання бази даних та створення інтерактивного інтерфейсу.

13.2. Програмні модулі та інтерфейси користувача

Система включає декілька ключових модулів, кожен з яких задокументовано відповідно до ролей користувачів:

- **Модуль «Каталог читача»:**
 - Забезпечує динамічний пошук та візуалізацію доступних книг.
 - Реалізовано систему карток із відображенням статусу примірника в реальному часі (Available, Reserved, Issued).
- **Модуль «Робоче місце бібліотекаря»:**
 - Інтерфейс для швидкої реєстрації видачі книг за ID користувача та інвентарним номером.
 - Автоматизоване оновлення статусу в базі даних після кожної транзакції.
- **Модуль «Адмін-панель та аналітика»:**
 - Використання бібліотеки Chart.js для побудови графіків активності та стану фонду.
 - Візуалізація співвідношення виданих книг до загальної кількості примірників.

13.3. Технічна документація API та бази даних

Система супроводжується специфікацією маршрутів (Endpoints), що забезпечують взаємодію рівнів:

- **Auth API:** /api/auth/login, /api/auth/register - обробка входу та безпечне хешування.
- **Library API:** /api/books, /api/loans - операції з книгами та транзакціями видачі.

Стан бази даних після розгортання: База даних PostgreSQL містить заповнені таблиці users, books, book_items та loans, що дозволяє системі функціонувати відразу після запуску. Цілісність даних підтверджена успішним виконанням тестових сценаріїв резервування та повернення.

13.4. Оцінка якості проєктного рішення

Проєкт відповідає заданим стандартам:

- **Масштабованість:** Архітектура дозволяє додавати нові модулі без рефакторингу ядра.
- **Безпека:** Впроваджено JWT та bcrypt для захисту чутливих даних.
- **Відповідність вимогам:** Всі 15 лекційних тем відображені у структурі документації та програмному коді.

РОЗДІЛ 14. ВИЗНАЧЕННЯ ЦИКЛІВ У СИСТЕМІ

Згідно з методологією системного аналізу, будь-яка складна інформаційна система функціонує як сукупність взаємопов'язаних контурів зворотного зв'язку. У проєкті ідентифіковано два ключові типи циклів, що визначають стабільність та розвиток бібліотеки.

14.1. Балансуючий цикл (Balancing Loop): Контроль книжкового фонду

Метою цього циклу є підтримка системи в стані рівноваги та забезпечення доступності ресурсів для читачів.

- **Механіка дії:** Збільшення кількості виданих книг призводить до зменшення доступного фонду.
- **Реакція системи:** Спрацьовує алгоритм overdue-check, який ідентифікує заборгованості та ініціює надсилання сповіщень боржникам.
- **Результат:** Повернення книг читачами збільшує кількість доступних примірників, повертаючи систему до цільового стану наповненості каталогу.

14.2. Підсилюючий цикл (Reinforcing Loop): Популярність та розвиток

Цей цикл відповідає за експоненціальне зростання активності в системі та її еволюцію.

- **Механіка дії:** Якісне наповнення каталогу та зручний інтерфейс пошуку залучають нових читачів.
- **Реакція системи:** Ріст кількості успішних бронювань та видач формує позитивну статистику в адмін-панелі.

- **Результат:** На основі високих показників активності адміністрація приймає рішення про розширення фонду та впровадження нових технологій (наприклад, IoT), що ще більше підвищує привабливість бібліотеки для нових користувачів.

14.3. Аналіз затримок (Delays) у циклах

У системі присутні часові затримки, які необхідно враховувати при проектуванні логіки:

- **Затримка між видачею та поверненням:** фізичний час перебування книги у читача, який регулюється параметром `due_date`.
- **Затримка сповіщення:** час між виявленням заборгованості сервером та моментом прочитання листа користувачем.

Розуміння цих циклів дозволяє не просто автоматизувати функції, а й керувати динамікою системи для запобігання «колапсу» (коли всі книги видані, а повернення не контролюється).

РОЗДІЛ 15. ВИЗНАЧЕННЯ ЯКОМУ АРТЕФАКТУ ВІДПОВІДАЄ СИСТЕМА

На основі методології системного проектування та аналізу лекційних матеріалів, розроблена інформаційна система класифікується як **Фактографічний управлінський артефакт (Management Information System - MIS)**.

15.1. Обґрунтування приналежності до артефактів

Згідно з класифікацією, наведеною у Лекції №3, система відповідає ключовим ознакам управлінського артефакту:

- **Цифровий двійник процесів:** Система не просто зберігає дані, а виступає точним відображенням реальних фізичних процесів бібліотеки (наявність книг на полицях, статус видачі, черга читачів).
- **Фактографічність:** Кожна транзакція в базі даних PostgreSQL (таблиця loans) є зафіксованим фактом події, що відбулася в об'єктній системі, із точною прив'язкою до часу та учасників.
- **Структурна цілісність:** Система має жорстку логічну структуру (ER-діаграма), де видалення або зміна одного елемента (наприклад, видалення книги) призводить до каскадних змін у пов'язаних сутностях, що характерно для складних інформаційних артефактів.

15.2. Функціональне призначення артефакту

Як управлінський артефакт, система виконує три фундаментальні функції:

1. **Ресстраційна:** фіксація станів об'єктів (книг та користувачів) у реальному часі.

2. **Контрольна:** виявлення відхилень від заданих параметрів (наприклад, перевищення терміну повернення `due_date`).
3. **Інформаційна:** надання адміністратору консолідованих даних для прийняття рішень щодо управління фондом.

15.3. Системна когерентність

Система демонструє властивість **когерентності**, оскільки всі її програмні модулі (від `AuthContext.js` до `AdminDashboard.js`) працюють узгоджено навколо єдиної мети - ефективного управління знаннями та активами бібліотеки. Це робить її не просто набором скриптів, а цілісним інтелектуальним артефактом, здатним підтримувати життєдіяльність організації.

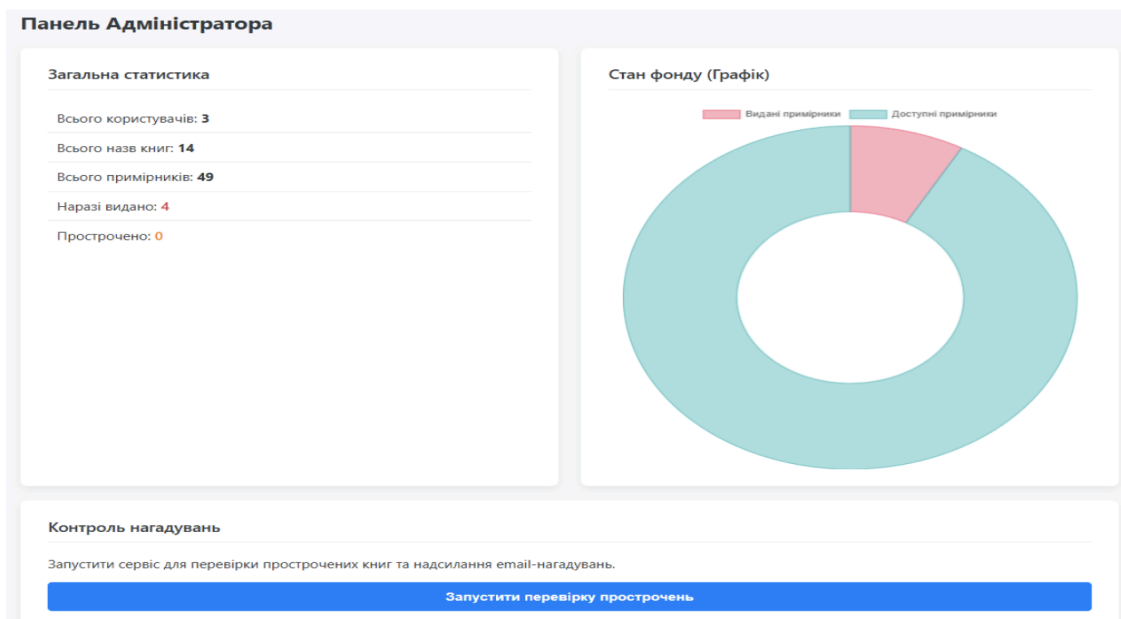
РОЗДІЛ 16. ЗНАЙТИ ТОЧКИ ВПЛИВУ НА СИСТЕМУ

Згідно з теорією системного аналізу (Лекція №2), **точки впливу (Leverage Points)** - це місця в складній системі, де незначне втручання може призвести до значних і сталих змін у поведінці всієї структури. Для інформаційної системи ідентифіковано декілька таких критичних вузлів.

16.1. Інформаційні потоки: Аналітична звітність

Однією з найпотужніших точок впливу є структура інформаційних потоків, реалізована в модулі AdminDashboard.js.

- **Механізм:** Візуалізація статистики видач та заборгованостей через Chart.js створює прозорість, якої не було при паперовому обліку.
- **Ефект:** Доступність цих даних у реальному часі дозволяє адміністрації миттєво змінювати стратегію закупівлі книг або переглядати правила користування фондом, що докорінно змінює ефективність бібліотеки.



Наразі видані книги (4)

Кладовище домашніх тварин(Прим. ID: 7)

Видано: Зоріна Софія Михайлівна (accidental.genius.1605@gmail.com)
Термін повернення: 03.11.2025

Фейрі й алхімія. Ртуть. Книга 1(Прим. ID: 32)

Видано: Зоріна Софія Михайлівна (accidental.genius.1605@gmail.com)
Термін повернення: 03.11.2025

Остання зупинка(Прим. ID: 48)

Видано: Зоріна Софія Михайлівна (accidental.genius.1605@gmail.com)
Термін повернення: 05.11.2025

Ніби ми злодії(Прим. ID: 41)

Видано: Зоріна Софія Михайлівна (accidental.genius.1605@gmail.com)
Термін повернення: 05.11.2025

Звіт про прострочені книги (0)

Прострочених книг немає. 🚩

Мал. №6

16.2. Правила системи: Параметризація термінів (due_date)

Зміна правил, за якими функціонує система, має високий важіль впливу на її стабільність.

- **Механізм:** Константа, що визначає тривалість видачі книги (наприклад, 14 або 30 днів), закладена в логіці контролера LoanController.js.
- **Ефект:** Незначне скорочення цього терміну для популярних книг різко пришвидшує обіг фонду, дозволяючи обслужити більше читачів тими ж ресурсами, що підсилює балансує цикл системи.

16.3. Контури зворотного зв'язку: Автоматизація сповіщень

Впровадження модуля overdue-check є точкою впливу на рівні підсилюючих та балансує циклів.

- **Механізм:** Автоматичне виявлення боржників та надсилання email-нагадувань.

- **Ефект:** Це мінімізує «затримку» (delay) між фактом протермінування та реакцією користувача, що запобігає вимиванню книжкового фонду та підтримує цілісність системи.

16.4. Парадигма системи: Перехід до самообслуговування

Найвищою точкою впливу є зміна самої мети або парадигми системи.

- **Механізм:** Впровадження функцій онлайн-резервування та особистого кабінету читача.
- **Ефект:** Це трансформує роль бібліотекаря з «контролера-наглядача» на «консультанта-навігатора», а читача - на активного учасника управління знаннями, що повністю змінює соціально-технічну динаміку закладу.

ВИСНОВОК ЩОДО ПОДАЛЬШОГО РОЗВИТКУ

Проект інформаційної системи продемонстрував успішну інтеграцію системного аналізу та сучасних веб-технологій. Розроблений артефакт є гнучкою платформою, яка завдяки трирівневій архітектурі та реляційній моделі даних готова до еволюційного розширення.

Пріоритетними кроками для майбутнього розвитку є впровадження мобільних інтерфейсів, IoT-ідентифікації та алгоритмів штучного інтелекту для прогнозування попиту, що дозволить системі перейти на вищий рівень інтелектуального управління інформаційними ресурсами.